RAPPORT DE TRAVAUX PRATIQUES

Administration d’un serveur Linux — Projet Hélios Logistique

Mise en production d’un portail de suivi des tournées

Module : Administration d’un Linux Server — BTC2

Environnement : Rocky Linux — srv-core et srv-edge

1. Présentation du projet

Le projet consiste à mettre en production un portail interne permettant aux exploitants d’Hélios Logistique de consulter l’état du collecteur de télémétrie des camions sans ouvrir de session SSH. Le portail repose sur le serveur HTTP intégré à Python et publie une page d’accueil ainsi qu’un fichier **etat.json**, régénéré toutes les deux minutes à partir de **/var/lib/helios/telemetry.csv**.

Le travail porte principalement sur l’administration et la sécurisation du service : comptes dédiés, permissions et ACL, scripts de publication et de sauvegarde, unités et timers systemd, pare-feu, supervision Zabbix et restauration après incident.

2. Démarche suivie

• J’ai commencé par vérifier l’état initial de srv-core : collecteur de télémétrie actif, cible helios.target présente, pare-feu en zone drop/internal, SSH toujours limité au port 2242 et aux membres de ssh-admins, et timer de surveillance d’authentification actif.

• J’ai installé en premier l’agent Zabbix et les outils nécessaires. Cette étape crée le compte système zabbix, indispensable pour configurer ensuite les ACL du répertoire publié.

• J’ai créé le groupe suivi, le compte lea et le compte de service helios-suivi, puis préparé l’arborescence /srv/helios/suivi/www avec les droits adaptés.

• J’ai ensuite développé et testé le script de publication, configuré les services et timers systemd, limité les droits sudo, ouvert le port 8080 uniquement dans la zone internal, puis ajouté la supervision et la sauvegarde.

• Enfin, j’ai prévu les contrôles de conformité, la restauration d’un fichier supprimé et la mesure de charge afin de vérifier le fonctionnement au-delà du simple démarrage du portail.

Précision : la fiche fournie est un cahier des charges et ne contient pas les sorties de terminal personnelles. Les résultats chiffrés, scores et preuves doivent être ajoutés à partir des commandes réellement exécutées ; aucune valeur n’est inventée dans ce rapport.

3. Réalisation technique et difficultés rencontrées

3.1 Comptes, répertoires et ACL

J'ai séparé les rôles pour appliquer le principe du moindre privilège : **lea** est l'exploitante, **helios-suivi** exécute la publication et le serveur HTTP, tandis que **zabbix** ne dispose que des accès nécessaires à ses contrôles. Le répertoire www est prévu avec le groupe suivi et le bit setgid afin que les nouveaux fichiers conservent le groupe attendu.

Difficulté : faire cohabiter les droits d'écriture de l'exploitante et du service avec l'accès très limité de Zabbix. Une permission classique trop large aurait permis de lister des fichiers ou d'exposer leur contenu.

Solution : utiliser les ACL pour accorder les droits explicitement. Pour Zabbix, le droit de traversée **--x** sur le répertoire permet d'atteindre un fichier connu et d'en lire les métadonnées avec stat, sans autoriser la liste du répertoire. Le contrôle attendu est que ls échoue pour zabbix tandis que stat sur etat.json réussit.

3.2 Script de publication

Le script **/usr/local/sbin/helios-suivi-publish.sh** lit la dernière mesure du fichier de télémétrie et génère un JSON valide contenant notamment l'horodatage, le nom de l'hôte, la dernière mesure, la charge moyenne, la mémoire disponible et le nombre de mesures.

Difficultés : éviter deux publications simultanées, empêcher qu'un lecteur voie un fichier JSON partiellement écrit et gérer les cas où la télémétrie ou le répertoire de destination est indisponible.

Solutions : utiliser un mode strict, un verrou exclusif, un nettoyage avec trap, une journalisation dédiée dans journald et des codes retour distincts (0 succès, 1 erreur interne, 2 verrou déjà pris, 3 prérequis manquant). La publication est écrite dans un fichier temporaire puis renommée pour garantir une mise à jour atomique.

3.3 Services et timers systemd

Le portail est lancé par **helios-suivi.service** sous le compte dédié, rattaché à helios.target. Le service est configuré pour redémarrer après un crash et respecter les plafonds de ressources MemoryMax=128M et CPUQuota=50%. La publication est exécutée par une unité oneshot déclenchée toutes les deux minutes.

Difficultés : le durcissement systemd peut empêcher un script de créer son verrou ou son fichier temporaire si le chemin n'est pas autorisé en écriture. Un service trop restreint peut également ne plus répondre correctement.

Solutions : limiter précisément les chemins inscriptibles avec ReadWritePaths, séparer le service HTTP (qui n'a pas besoin d'écrire) du service de publication, puis vérifier le fonctionnement avec systemctl, curl et systemd-analyze security. Le score réellement obtenu est à reporter : **[à compléter]/10**.

4. Sécurité réseau, délégation et supervision

4.1 Pare-feu et conservation de la sécurité SSH

Le portail doit être accessible depuis le réseau métier uniquement. Le port TCP 8080 est donc autorisé dans la zone **internal**, en configuration runtime et permanente. La zone par défaut reste **drop**. Depuis srv-edge, l'adresse métier 10.42.0.10:8080 doit répondre, alors que l'accès à 192.168.56.10:8080 depuis le réseau d'administration doit échouer.

Difficulté : ouvrir le nouveau service sans exposer le serveur sur les autres interfaces et sans dégrader les règles existantes. Le projet impose également de ne pas modifier la configuration SSH héritée des TP précédents.

Solution : ajouter uniquement la règle 8080/tcp à la zone internal et contrôler les autres zones. Après modification, comparer sshd -T : port 2242, PermitRootLogin no, PasswordAuthentication no et AllowGroups ssh-admins. Le compte lea ne doit pas obtenir d'accès SSH.

4.2 Délégation sudo

L'exploitante doit pouvoir redémarrer le portail, afficher son état, déclencher une publication et consulter les 100 dernières lignes des journaux des deux unités. Elle ne doit pas disposer de droits d'administration généraux.

Difficulté : une règle sudo trop large, notamment avec un joker * derrière journalctl, pourrait autoriser des options supplémentaires et des actions non prévues.

Solution : définir des commandes et arguments précis dans /etc/sudoers.d/92-helios-suivi, valider le fichier avec visudo -c, vérifier les droits du fichier (0440 root:root), puis contrôler les autorisations avec sudo -l -U lea. Aucun NOPASSWD ni droit de redémarrer sshd ne doit être accordé.

4.3 Supervision Zabbix et collecte système

L'agent zabbix-agent2 est configuré localement avec deux sondes : **helios.suivi.http** renvoie le code HTTP du fichier etat.json (attendu : 200) et **helios.suivi.age** renvoie son âge en secondes (attendu : moins de 300). Le timer sysstat permet de collecter les statistiques système.

Difficulté : les contrôles doivent fonctionner sous l'identité zabbix, qui ne doit pas avoir accès à la liste des fichiers du portail.

Solution : adapter les ACL, puis tester les clés avec zabbix_get et vérifier que l'agent ainsi que sysstat-collect.timer sont actifs et activés au démarrage.

4.4 Infrastructure réseau déjà préparée

Les éléments construits dans les TP précédents — bond réseau, VLAN 42, routage et persistance de la configuration réseau — constituent le socle de l'environnement. Pour ce projet, l'objectif est de les conserver sans régression : le portail ajoute une règle de pare-feu ciblée, sans remplacer la configuration réseau existante.

Preuves à joindre : firewall-cmd --zone=internal --list-all, tests curl/nc depuis srv-edge, sshd -T, sudo -l -U lea et résultats zabbix_get. Insérer les sorties réellement obtenues dans le dossier preuves/.

5. Sauvegarde, restauration et tests

5.1 Sauvegarde automatisée

Le script **/usr/local/sbin/helios-suivi-backup.sh** archive les éléments nécessaires à la reconstruction du portail : répertoire /srv/helios/suivi, unités systemd du projet, fichier sudoers, configuration Zabbix et scripts. L'archive inclut les ACL et est accompagnée d'une empreinte SHA-256. Le timer lance la sauvegarde chaque jour à 03 h 15 et doit rattraper une exécution manquée après un arrêt du serveur. Seules les sept archives les plus récentes sont conservées.

Difficultés : une archive du seul contenu web ne suffirait pas à reconstruire le service ; les unités et fichiers de configuration doivent aussi être sauvegardés. Il faut également conserver les ACL pour restaurer les permissions particulières.

Solutions : inclure tous les composants du service dans l'archive, utiliser tar avec la prise en charge des ACL, générer un fichier .sha256, vérifier l'intégrité avec sha256sum -c et appliquer une rotation limitée aux sept dernières sauvegardes.

5.2 Procédure de restauration

La restauration est vérifiée en deux temps. Premièrement, extraire la dernière archive dans un répertoire temporaire et comparer les ACL du répertoire www restauré avec celles de la production. Deuxièmement, simuler un incident en supprimant index.html, puis restaurer uniquement ce fichier depuis l'archive.

```
sha256sum -c /srv/helios/archives/suivi/<archive>.sha256
mkdir -p /tmp/helios-restauration
tar --acls -xzf /srv/helios/archives/suivi/<archive>.tar.gz -C /tmp/helios-restauration
getfacl /srv/helios/suivi/www
rm /srv/helios/suivi/www/index.html
tar --acls -xzf /srv/helios/archives/suivi/<archive>.tar.gz -C / -- <chemin-dans-archive>/index.html
```

Après restauration, vérifier la présence du fichier, son contenu et ses ACL avec ls -l et getfacl. Le chemin exact dans l'archive doit être relevé avec tar -tzf avant l'extraction ciblée. **Résultat et durée réellement constatés :** [à compléter après le test].

5.3 Mesure des performances

Le test consiste à envoyer 2 000 requêtes locales vers etat.json et à relever les compteurs CPU et mémoire avant et après. sar -u est exécuté pendant la fenêtre de charge.

```
systemctl show -p CPUUsageNSec -p MemoryCurrent helios-suivi.service
sar -u 1 5
for i in $(seq 2000); do curl -s -o /dev/null http://127.0.0.1:8080/etat.json; done
```

Les mesures doivent permettre de dire si le quota CPUQuota=50% a réellement été atteint. Le quota est une limite maximale et non une consommation garantie. **Mesures :** CPU avant/après [à compléter] ; mémoire avant/après [à compléter] ; utilisation CPU pendant le test [à compléter] ; quota atteint ou non [à conclure à partir des chiffres].

6. Tableau récapitulatif des contrôles

Exigence	Contrôle principal	Preuve à renseigner
E1 Comptes	id lea ; getent passwd ; chage -l lea	[sorties réelles]
E2 ACL	ls -ld ; getfacl ; tests runuser	[sorties réelles]
E3 Publication	script ; json.tool ; journalctl -t	[codes 0, 2 et 3]
E4 systemd	systemctl status/show ; curl ; security	[état + score]
E5 Timer	systemctl list-timers ; date du JSON	[exécution 2 min]
E6 Sudo	visudo -c ; sudo -l -U lea	[autorisations/refus]
E7 Pare-feu/SSH	firewall-cmd ; curl/nc ; sshd -T	[tests réseau + SSH]
E8 Supervision	zabbix_get ; systemctl status	[HTTP 200 ; âge <300 s]
E9 Sauvegarde	sha256sum -c ; test de restauration	[empreinte + restauration]
E10 Bonus	helios-suivi-audit.sh	[Bilan n/9 si réalisé]

7. Limites connues et améliorations

• Le serveur HTTP intégré à Python convient à ce portail de laboratoire et à un usage interne limité, mais ne remplace pas une solution web complète exposée à grande échelle. Une évolution pourrait intégrer un serveur web dédié et TLS.

• Le redémarrage systemd protège contre l'arrêt du processus, mais pas contre une panne complète de la machine ou du stockage. Une copie des sauvegardes sur un support distinct améliorerait la reprise.

• La supervision vérifie le code HTTP et l'ancienneté du JSON. Des alertes supplémentaires pourraient porter sur l'espace disque, les erreurs du script, l'état des sauvegardes et la fraîcheur de la télémétrie.

• Les tests de restauration et de charge doivent être rejoués périodiquement et leurs preuves datées conservées dans le dépôt.

Conclusion

Ce projet m'a permis de réunir les compétences travaillées dans les TP précédents : gestion des comptes et permissions, ACL, scripts d'administration, services et timers systemd, sécurisation réseau, supervision et sauvegarde. La démarche a consisté à construire chaque composant séparément, à contrôler ses droits et son fonctionnement, puis à vérifier leur intégration sur le serveur. La mise en production ne peut être déclarée totalement conforme qu'après validation des exigences E1 à E9 avec les sorties réelles, la restauration effective d'index.html et l'analyse des mesures sous charge.

À finaliser avant le rendu : compléter les champs entre crochets avec les résultats réellement obtenus, ajouter les sorties de terminal utiles et vérifier que les services et règles des TP1 à TP3 sont toujours actifs. La fiche de projet précise qu'une sortie inventée ou retouchée ne doit pas être remise.